UNIVERSIDAD DON BOSCO

Primer Proyecto (React)

Materia: Diseño y Programación de Software Multiplataforma DPS941 G01T (Virtual)

Docente: Ing. Alexander Alberto Siguenza Campos

Integrantes:

Nombres	Apellidos	Carné
Diego Guillermo	Esnard Romero	ER231474
Eduardo Ezequiel	López Rivera	LR230061
Diego René	López Martínez	LM231893
Christian Gustavo	Crespin Lozano	CL060107
Andrés René	Velásquez Rodríguez	VR222732

Fecha de Entrega: 22 de marzo de 2026

Índice

Índice 1
1. Enlace del repositorio 2
2. Introducción 3
3. Objetivos 4
3.1 Objetivo General 4
3.2 Objetivos Específicos 4
4. Descripción general del proyecto 5
5. Tecnologías utilizadas 6
6. Arquitectura y estructura del sistema 8
7. Funcionalidades principales 9
8. Control de acceso y manejo de roles 11
9. Proceso de ejecución e instalación 12
10. Aportes realizados en el desarrollo 13
11. Dificultades encontradas y solución aplicada 14
12. Conclusiones 17
13. Referencias 18

1. Enlace del repositorio

A continuación, se presenta el enlace del repositorio oficial del proyecto, alojado en GitHub. En este espacio se gestionó el código fuente de la aplicación, permitiendo organizar el desarrollo, registrar los cambios realizados y facilitar el trabajo colaborativo entre los integrantes del equipo.

El repositorio contiene la estructura general del sistema, los archivos correspondientes a cada módulo desarrollado y el historial de aportes efectuados durante la construcción del proyecto.

Repositorio del proyecto: https://github.com/eduardoezequieel/proyecto1_dps

2. Introducción

El presente documento describe el desarrollo del Primer Proyecto en React, elaborado en la asignatura Diseño y Programación de Software Multiplataforma DPS941. El proyecto consistió en la creación de una aplicación web orientada a la gestión de proyectos y tareas, con el propósito de aplicar conocimientos fundamentales de desarrollo frontend, estructuración de interfaces, manejo de componentes y organización lógica de un sistema web moderno.

La solución fue construida con tecnologías actuales del ecosistema JavaScript, principalmente React y Next.js, integrando además herramientas como TypeScript, Tailwind CSS y json-server para simular el comportamiento de una API. De esta manera, el proyecto permitió acercarse a un entorno de desarrollo más práctico, en el que no solo se trabajó la parte visual de la aplicación, sino también la navegación, la autenticación básica por roles y la interacción con datos simulados.

A lo largo del desarrollo, se aplicó una metodología de trabajo colaborativa basada en ramas dentro de GitHub, permitiendo que cada integrante asumiera una responsabilidad específica dentro del sistema. Esto favoreció la organización del equipo, la distribución de tareas y la integración progresiva de los distintos módulos del proyecto.

3. Objetivos

3.1 Objetivo General

Desarrollar una aplicación web de gestión de proyectos y tareas utilizando React y tecnologías complementarias, con el fin de aplicar de forma práctica los conocimientos adquiridos en la asignatura sobre desarrollo de software multiplataforma.

3.2 Objetivos Específicos

- Diseñar una interfaz web funcional y organizada para la administración de proyectos y tareas.
- Implementar un sistema básico de autenticación con control de acceso según el rol del usuario.
- Utilizar componentes reutilizables para mantener una estructura más clara y ordenada dentro de la aplicación.
- Integrar una API simulada mediante json-server para manejar datos de usuarios, proyectos y tareas.
- Aplicar el uso de GitHub y ramas de trabajo para fomentar el desarrollo colaborativo y la integración de cambios.
- Fortalecer habilidades prácticas en el uso de React, Next.js, TypeScript y Tailwind CSS dentro de un proyecto realista.

4. Descripción general del proyecto

El proyecto desarrollado consiste en una aplicación web orientada a la gestión de proyectos y tareas, diseñada para apoyar la organización de actividades dentro de un entorno de trabajo simulado. Su propósito principal es permitir que los usuarios puedan visualizar información relevante, administrar proyectos, dar seguimiento a tareas y mantener un control más claro sobre el estado general del trabajo.

La aplicación fue construida como una solución frontend moderna, empleando React junto con Next.js para estructurar la navegación y la interfaz. Además, se incorporó una API simulada mediante json-server, lo que permitió trabajar con datos de usuarios, proyectos y tareas de una manera cercana a un entorno real de desarrollo.

Dentro de la solución se contempló un acceso básico al sistema mediante autenticación por roles, diferenciando principalmente entre usuarios con mayores privilegios de administración y usuarios con funciones más operativas. Esto permitió establecer una lógica inicial de control de acceso y protección de rutas dentro de la aplicación.

De forma general, el sistema incluye módulos como inicio de sesión, panel principal, administración de proyectos, gestión de tareas y visualización de usuarios. Cada una de estas partes fue distribuida entre los integrantes del equipo, siguiendo una estrategia de trabajo por ramas para facilitar la integración progresiva del sistema.

En conjunto, este proyecto representa una práctica integral de desarrollo web, ya que no solo abarca el diseño visual de la interfaz, sino también aspectos importantes como estructura del código, organización modular, simulación de servicios, trabajo colaborativo y control de versiones.

5. Tecnologías utilizadas

Para el desarrollo del proyecto se utilizaron diversas tecnologías y herramientas que permitieron construir una aplicación web moderna, organizada y funcional. Cada una de ellas cumplió un papel importante dentro de la estructura del sistema y en el proceso de desarrollo colaborativo.

En primer lugar, se utilizó React como biblioteca principal para la construcción de la interfaz de usuario. Su enfoque basado en componentes facilitó la creación de vistas reutilizables y una mejor organización del código. Junto a ello, se trabajó con Next.js, el cual permitió estructurar la aplicación de una forma más ordenada, manejar rutas y aprovechar una base moderna para el desarrollo web.

También se empleó TypeScript, con el fin de mejorar el control sobre los datos manejados por el sistema y reducir errores mediante el uso de tipado estático. Esta herramienta aportó

mayor claridad al código y una mejor definición de las estructuras utilizadas en usuarios, proyectos y tareas.

Para el diseño visual de la aplicación se utilizó Tailwind CSS, una herramienta que permitió aplicar estilos de forma rápida y mantener una interfaz limpia, consistente y adaptable. Gracias a ello, fue posible construir pantallas con una apariencia moderna sin depender de archivos de estilos excesivamente extensos.

En la parte de simulación de datos se utilizó json-server, el cual funcionó como una API falsa o entorno backend simulado. Esto permitió trabajar con registros de usuarios, proyectos y tareas de manera práctica, realizando consultas y operaciones básicas similares a las de un sistema real.

Asimismo, se usó Axios para realizar las solicitudes a la API simulada, facilitando la conexión entre la interfaz y los datos. Para el manejo de formularios y validaciones también se integraron herramientas complementarias como React Hook Form y Zod, útiles para controlar entradas de datos de manera más ordenada.

Finalmente, para el control de versiones y el trabajo colaborativo del equipo se utilizó GitHub, mediante el uso de repositorios y ramas de desarrollo. Esto permitió distribuir responsabilidades, mantener un historial de cambios y realizar la integración progresiva de cada módulo desarrollado.

6. Arquitectura y estructura del sistema

La arquitectura del proyecto fue organizada de forma modular, con el objetivo de mantener una separación clara entre las distintas responsabilidades del sistema. Esta estructura permitió distribuir mejor el trabajo entre los integrantes del equipo y facilitó tanto el desarrollo como la integración de cada parte de la aplicación.

En términos generales, el sistema se compone de una capa de interfaz construida con React y Next.js, una capa de lógica para el manejo de autenticación, roles y consumo de datos, y una capa de datos simulada mediante json-server. Aunque no se trabajó con un backend real, la organización del proyecto permitió aproximarse a una estructura similar a la que se utiliza en aplicaciones modernas.

Dentro del proyecto, los archivos fueron distribuidos en carpetas según su función. Por ejemplo, la carpeta de rutas o páginas concentra las vistas principales del sistema, mientras que otras carpetas reúnen componentes reutilizables, contextos, servicios y tipos. Esta separación ayudó a que cada módulo tuviera un espacio definido y a que el código fuera más legible y mantenible.

La carpeta de componentes fue utilizada para encapsular elementos visuales y funcionales que podían reutilizarse en distintas partes del sistema. La carpeta de contexto permitió centralizar información importante, como la autenticación del usuario. Por su parte, la

carpeta de servicios se encargó de gestionar la comunicación con la API simulada, y la carpeta de tipos reunió las definiciones necesarias para trabajar con TypeScript de manera más ordenada.

Además, el sistema fue diseñado considerando una estructura por módulos funcionales, como inicio de sesión, panel principal, proyectos, tareas y usuarios. Esta organización no solo mejoró la comprensión general del proyecto, sino que también permitió que cada integrante desarrollara una parte específica sin afectar directamente el trabajo de los demás.

En conjunto, la arquitectura implementada favoreció un desarrollo más limpio, comprensible y escalable, sentando una base adecuada para futuras mejoras o ampliaciones del sistema.

7. Funcionalidades principales

La aplicación desarrollada incorpora un conjunto de funcionalidades orientadas a la gestión básica de proyectos y tareas dentro de un entorno web. Estas funciones fueron diseñadas para simular el comportamiento de una herramienta administrativa, permitiendo al usuario interactuar con la información de manera clara y ordenada.

Una de las funcionalidades principales es el inicio de sesión, mediante el cual se valida el acceso al sistema según el tipo de usuario registrado. Esta parte permite identificar el rol del usuario y habilitar el acceso correspondiente dentro de la aplicación.

Otra función importante es la visualización del panel principal o dashboard, donde se presenta una vista general del sistema. Desde este espacio, el usuario puede acceder a los módulos principales y tener una mejor orientación sobre la información disponible.

El sistema también incluye la gestión de proyectos, que permite consultar, crear, editar y eliminar proyectos según la lógica implementada. Esta funcionalidad facilita llevar un control general de los proyectos registrados, incluyendo datos relevantes como nombre, descripción, prioridad, estado y fechas asociadas.

De igual forma, se incorporó la gestión de tareas, mediante la cual se pueden administrar actividades vinculadas a los proyectos. Esta parte del sistema permite dar seguimiento a las tareas asignadas, su estado, prioridad y otros datos necesarios para el control del trabajo.

Otra funcionalidad relevante es la visualización de usuarios, útil para identificar a las personas registradas dentro del sistema y relacionarlas con determinadas acciones, tareas o proyectos. Esto contribuye a una organización más clara dentro del entorno de trabajo simulado.

Finalmente, la aplicación cuenta con protección de rutas y control de acceso, lo que impide que un usuario ingrese libremente a determinadas secciones sin haber iniciado sesión. Esta función fortalece la lógica básica de seguridad y da mayor coherencia al comportamiento general del sistema.

8. Control de acceso y manejo de roles

El sistema incorpora un control de acceso básico que permite restringir el ingreso a determinadas secciones de la aplicación según el estado de autenticación del usuario. Esta funcionalidad fue incluida con el propósito de brindar mayor orden al flujo del sistema y simular un comportamiento más cercano al de una aplicación real.

Para acceder a las vistas protegidas, el usuario debe iniciar sesión con credenciales registradas dentro de la base de datos simulada. Una vez validada la información, el sistema almacena los datos principales del usuario y habilita su navegación dentro de la plataforma. Si el usuario no ha iniciado sesión, no puede ingresar libremente a las rutas restringidas.

Además del control de acceso general, el sistema maneja roles de usuario para diferenciar el tipo de interacción permitida dentro de la aplicación. En este proyecto se trabajó principalmente con dos roles: gerente y usuario. Esta distinción permite representar diferentes niveles de responsabilidad dentro del sistema.

El rol de gerente se asocia con funciones de mayor control dentro de la aplicación, mientras que el rol de usuario representa un perfil con participación más limitada y operativa. Aunque se trata de una implementación básica, esta separación permite sentar una base importante para futuros niveles de permisos o reglas de autorización más detalladas.

También se implementó protección de rutas, evitando que páginas sensibles puedan abrirse sin validación previa. Esto ayuda a mantener la coherencia del sistema, mejora la

experiencia de uso y refuerza la lógica de seguridad del proyecto desde una perspectiva funcional.

9. Proceso de ejecución e instalación

Para ejecutar correctamente el proyecto, primero fue necesario contar con un entorno de desarrollo con Node.js y un editor de código, como Visual Studio Code. Una vez preparado el entorno, el equipo trabajó con el repositorio compartido en GitHub, desde donde cada integrante descargó el proyecto y realizó sus respectivos aportes en una rama específica.

Después de clonar el repositorio, se instalaron las dependencias necesarias mediante el gestor de paquetes de Node.js. Este paso permitió incorporar todas las librerías requeridas por la aplicación, como React, Next.js, TypeScript, Tailwind CSS, Axios y otras herramientas complementarias utilizadas durante el desarrollo.

Posteriormente, para poner en funcionamiento el sistema, fue necesario ejecutar tanto la aplicación principal como la API simulada. La aplicación web se levantó en un servidor local de desarrollo, mientras que la base de datos simulada se ejecutó mediante json-server, permitiendo cargar y consultar información de usuarios, proyectos y tareas.

El proceso de instalación y ejecución también implicó verificar la estructura correcta de carpetas y archivos, especialmente en los módulos relacionados con autenticación, rutas y

consumo de datos. Esto fue importante para asegurar que la aplicación reconociera correctamente las vistas, los componentes y los servicios definidos en el proyecto.

Una vez completados estos pasos, el sistema pudo ejecutarse de forma local en el navegador, permitiendo validar el inicio de sesión, la navegación entre módulos y el funcionamiento general de la solución desarrollada. Este proceso resultó clave para realizar pruebas, identificar errores y comprobar que la integración de los distintos aportes del equipo se hubiera realizado de manera correcta.

10. Aportes realizados en el desarrollo

El desarrollo del proyecto se llevó a cabo de forma colaborativa, distribuyendo responsabilidades específicas entre los integrantes del equipo. Esta forma de trabajo permitió que cada participante se enfocara en una parte concreta del sistema, facilitando el orden en el desarrollo y reduciendo conflictos al momento de integrar los cambios.

Dentro de esta dinámica, los aportes realizados abarcaron distintas áreas importantes de la aplicación, como la estructura base del proyecto, el diseño de la interfaz, la autenticación, la gestión de proyectos, la administración de tareas y la organización de usuarios. Gracias a esta distribución, fue posible avanzar de manera progresiva en la construcción del sistema y mantener una mejor coordinación entre los módulos.

En el caso particular del aporte realizado, se trabajó principalmente en la parte relacionada con la infraestructura de autenticación y control de acceso. Esto incluyó la integración de archivos clave para el manejo del inicio de sesión, la conexión con la API simulada, la definición de tipos base, la configuración del contexto de autenticación y la protección de rutas dentro de la aplicación.

También fue necesario revisar la estructura del proyecto para asegurar que los archivos estuvieran ubicados correctamente y que la aplicación pudiera reconocer las rutas y componentes implementados. Este proceso implicó pruebas, validaciones y ajustes técnicos para lograr que la funcionalidad desarrollada pudiera integrarse de manera adecuada con el resto del sistema.

Además del aporte técnico, el trabajo realizado contribuyó al proceso general de integración del proyecto, ya que permitió dejar funcional una parte esencial del sistema: el acceso inicial a la plataforma. Esta sección representa una base importante para el uso del resto de módulos, debido a que controla el ingreso de usuarios y establece una lógica inicial de seguridad y organización dentro de la aplicación.

11. Dificultades encontradas y solución aplicada

Durante el desarrollo del proyecto se presentaron algunas dificultades técnicas y de organización que fueron resolviéndose de manera progresiva. Estas situaciones formaron parte natural del proceso de construcción de la aplicación y, al mismo tiempo, permitieron fortalecer el aprendizaje práctico del equipo.

Una de las principales dificultades estuvo relacionada con la estructura del proyecto, ya que al inicio existía confusión sobre la ubicación correcta de ciertos archivos y carpetas. Esto generó problemas al momento de intentar ejecutar algunas rutas, especialmente en la parte del inicio de sesión, debido a que la aplicación no reconocía correctamente ciertos módulos dentro de la estructura esperada.

Otra dificultad importante fue la integración de los aportes realizados por cada integrante, ya que cada módulo debía incorporarse sin afectar el funcionamiento del resto del sistema. Para resolver esto, fue necesario trabajar con mayor cuidado en la organización por ramas, revisar la asignación de archivos y validar que cada cambio se integrara en la carpeta correspondiente.

También se presentaron inconvenientes durante la ejecución local del proyecto, particularmente al levantar de forma simultánea la aplicación principal y la API simulada. En este punto fue necesario revisar comandos, dependencias, rutas y archivos base para lograr que tanto el frontend como json-server funcionaran de manera conjunta.

En cuanto a la parte desarrollada, una de las dificultades más relevantes fue lograr que el módulo de autenticación y la ruta de inicio de sesión se integraran correctamente dentro de la estructura del sistema. La solución aplicada consistió en revisar la ubicación de los archivos, ajustar la organización del proyecto y realizar pruebas continuas hasta confirmar que la aplicación pudiera reconocer el acceso, la navegación y la protección de rutas de manera adecuada.

En general, cada dificultad encontrada fue abordada mediante revisión del código, pruebas locales, ajustes de estructura y trabajo coordinado entre los integrantes. Esto permitió no solo resolver problemas concretos, sino también comprender mejor la importancia de la organización, la integración progresiva y la validación constante en el desarrollo de software.

12. Conclusiones

El desarrollo de este proyecto permitió aplicar de manera práctica los conocimientos adquiridos en la asignatura, especialmente en temas relacionados con la construcción de interfaces web, organización modular del código y trabajo colaborativo en equipo. Más allá del resultado final, el proceso de desarrollo fue una experiencia importante para comprender cómo se estructura una aplicación moderna basada en componentes.

Asimismo, el proyecto ayudó a fortalecer habilidades técnicas en el uso de herramientas actuales como React, Next.js, TypeScript y Tailwind CSS, así como en la integración de una API simulada para el manejo de datos. Esto permitió acercarse a una dinámica de desarrollo más realista y comprender mejor la relación entre interfaz, lógica y datos.

Otro aspecto valioso fue el trabajo colaborativo mediante GitHub y el uso de ramas, ya que esto favoreció la distribución de responsabilidades y la integración progresiva de los distintos módulos. Gracias a esta metodología, fue posible organizar mejor el avance del proyecto y reducir conflictos al momento de unir los aportes de cada integrante.

En conclusión, este proyecto no solo permitió cumplir con los requerimientos académicos planteados, sino también desarrollar una visión más completa sobre el proceso de creación de software web. La experiencia obtenida representa una base importante para futuros proyectos de mayor complejidad dentro del área del desarrollo multiplataforma.

13. Referencias

- Universidad Don Bosco. (2026). *Unidad 1* [Material didáctico del curso *Diseño y Programación de Software Multiplataforma DPS941 G01T (Virtual)*]. Aula Digital.
- Universidad Don Bosco. (2026). *Unidad 2* [Material didáctico del curso *Diseño y Programación de Software Multiplataforma DPS941 G01T (Virtual)*]. Aula Digital.
- Meta Platforms, Inc. (s. f.). *React*. React. Recuperado el 21 de marzo de 2026, de https://react.dev/
- Vercel. (s. f.). *Next.js documentation*. Next.js. Recuperado el 21 de marzo de 2026, de https://nextjs.org/docs
- Microsoft. (s. f.). *TypeScript documentation*. TypeScript. Recuperado el 21 de marzo de 2026, de https://www.typescriptlang.org/docs/
- Tailwind Labs. (s. f.). *Tailwind CSS documentation*. Tailwind CSS. Recuperado el 21 de marzo de 2026, de https://tailwindcss.com/docs